



22.503 · Programación para la ciencia de datos

Grado en Ciencia de Datos Aplicada

Estudios de Informática, Multimedia y
Telecomunicación

Programación para la ciencia de datos - PEC4

En este Notebook encontraréis el ejercicio que supone la cuarta y última actividad de evaluación continuada (PEC) de la asignatura. Esta PEC intenta presentaros un pequeño proyecto en el cual debéis resolver diferentes ejercicios, que engloba muchos de los conceptos cubiertos durante la asignatura.

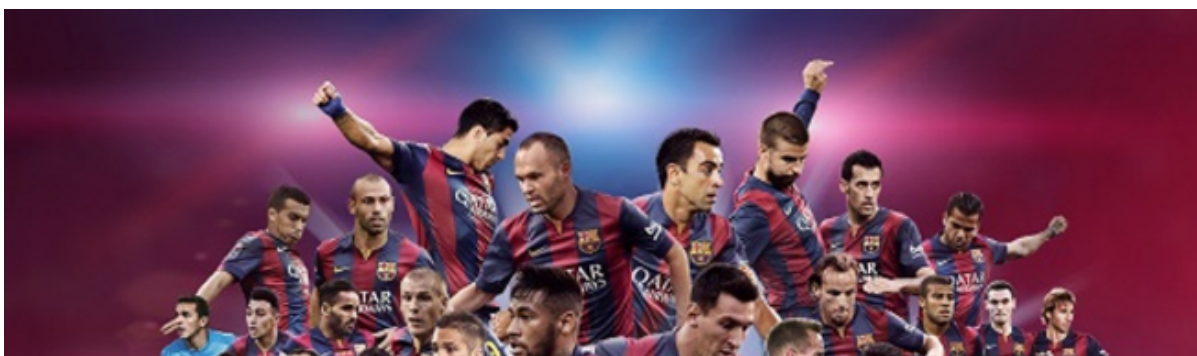
El objetivo de este ejercicio será desarrollar un **proyecto Python** fuera del entorno de Notebooks, que nos permita resolver el problema dado. Trabajaréis en archivos Python planos (`.py`). Éste tendrá que incluir el correspondiente código organizado lógicamente (separado por módulos, organizados por funcionalidad,...), la documentación del código (*docstrings*) y tests. Además, tendréis que incluir los correspondientes archivos de documentación de alto nivel (`README`) así como los archivos de licencia y dependencias (`requirements.txt`) comentados en la teoría.

Hacer un `setup.py` es opcional, pero si se hace se valorará positivamente de cara a la nota de la práctica y del curso.

✓ Liga de fútbol 1995-2025

En esta PEC4 analizaremos los datos históricos de la Liga española de fútbol, desde 1995 hasta 2025. El dataset lo hemos extraído de:

- <https://www.kaggle.com/datasets/kishan305/la-liga-results-19952020>





La PEC4 consta de unos 7 primeros ejercicios relacionados con el dataset.

NOTA: Estos ejercicios los puedes realizar de forma provisional en un cuaderno de Jupyter, si lo crees conveniente (por la ventaja de poder ejecutar las celdas individualmente).

Una vez realizados los 7 ejercicios, se deberá dar forma de proyecto Python a todo el código, y estos son los aspectos que consideramos importantes: trabajar con módulos, generación de documentación, tests, calidad del código, GitHub, etc. (más información al final).

En el código deberás definir dos variables globales:

```
nom_alumne = "nom_alumne" # substituir por el nombre del  
alumne date_time = datetime.now().strftime('%Y%m%d_%H%M%S')
```

de manera que cada vez que generes una imagen/gráfica, esta se guardará en la carpeta `img/` con el nombre:

```
plt.savefig(f"img/grafica_exX_{config.nom_alumne}  
_{config.date_time}.png")
```

En la carpeta `img/` se espera encontrar todas las gráficas que se solicitan, con el nombre del alumno y el timestamp del momento en que se generaron.

Las funciones que tendrás que definir no contendrán `print()` (excepto, quizás, en el primer ejercicio).

En esta PEC4 se te pide utilizar Type Hints, como has hecho en las otras PECs. Recuerda utilizar rutas relativas.

Ej 1. (0,4p) Carega del dataset y análisis exploratorio de los datos

La descripción de las columnas del dataset es:

```

FTHG - Number of goals scored by Home Team.
FTAG - Number of goals scored by Away Team.
FTR - Full time result.
HTHG - Number of goals scored by Home Team at Half time.
HTAG - Number of goals scored by Away Team at Half time.
HTR - Half time result.
H - Home Team.
A - Away Team.
D - Draw.

```

- Define la función `load_and_eda(file)`, que carga el dataset y elimina las columnas "HTHG", "HTAG", "HTR". La función también muestra los primeros y últimos valores del dataset, y la información más relevante.
- Ejecuta la función.
- Define la función `plot_home_away_goals(data)`, que contiene una figura con dos plots. En la gráfica se muestra cómo es la distribución de goles marcados por los equipos de casa y por los equipos de fuera. Ayuda: puedes utilizar `plt.boxplot`
- Ejecuta la función

Ej 2. (0,6p) Partidos totales jugados

- define la función `total_matches(data)`, que devuelve un dataframe `matches_team_total` con el número de partidos jugados por cada equipo durante todos los años.
- ejecuta la función y muestra los 10 primeros valores.
- muestra los equipos que siempre han estado en primera división (los que tienen el máximo número de partidos).
- define la función `plot_matches_team_total(matches_team_total)`, que representa esta información (eje x: equipos; eje y: número de partidos total). Ejecuta la función.

Ej 3. (0,6p) Distribución de goles

- define la función `goals_distribution(data)`, que devuelve dos dataframes: `distr_goals_home`, `distr_goals_away`. Estos dataframes contienen como índice el número de goles marcados (por ej, FTHG), y como columna el número de partidos en que se han marcado este número de goles. Por ejemplo, hay 2598 partidos en que FTHG=0 (los equipos locales no marcaron ningún).
Ejecuta la función y muestra el dataframe resultante.

- ejecuta la función y muestra el dataframe resultante.
- define la función `plot_goals_distribution(distr_goals_home, distr_goals_away)` para representar esta información (una gráfica, dos plots). Ejecuta la función.

Ej 4. (0,6p) Partidos ganados en casa/fuera

- Define la función `FTR(data)`, que devuelve el número de partidos que han ganado los locales, que han ganado los visitantes, o que han empatado (`draw`), en forma de dataframe (dataframe `ftr`).
- Ejecuta la función y muestra el dataframe resultante.
- ¿Qué porcentaje de partidos ganan los locales?
- Define la función `plot_FTR(ftr)` que representa esta información (eje x: H, A, D; eje y: número de partidos). Ejecuta la función.

Ej 5. (0,6p) Clasificación global 1995-2025

- define la función `add_points(data)`, que añade al dataset los puntos conseguidos como local y como visitante, por cada partido. Es decir, crea las columnas `points_home` y `points_away`, que pueden coger los valores de 3, 1 o 0.
- ejecuta la función y muestra los 10 primeros valores.
- define la función `fun_total_points(data)`, que calcula el total de puntos conseguidos y acumulados desde 1995 por cada equipo. Devuelve por cada equipo el número de puntos totales, en forma de tupla: variable Series y variable Dataframe (el Series y el Dataframe de hecho contienen la misma información). (Es una propuesta, un alumno puede decidir que sólo devuelva la Series o el Dataframe).
- ejecuta la función y muestra los 10 primeros valores.
- define la función `alltime_winner(df_total_points)`, que devuelve el ganador de esta liga histórica y acumulada (el equipo que ha acumulado más puntos). Ejecuta la función y muestra el ganador histórico.

Ej 6. (0,6p) Dataframe summary_1995_2025.

Podium.

- Define la función `fun_total_goals(data)`, que devuelve una tupla de tres números enteros: (`home_goals`, `away_goals`, `total_goals`). Muestra esta

información.

- Define la función *fun_total_goals_by_team(data)*, que devuelve una tupla de tres dataframes: *home_goals_by_team*, *away_goals_by_team*, *total_goals_by_team*. Muestra los 10 primeros valores de *total_goals_by_team*.
- En el ejercicio 5 calculaste el número total de puntos por equipo:

```
total_points_by_team, df_total_points_by_team =  
fun_total_points(data)
```
- Define la función *fun_summary_1996_2025(, , ,)* que crea el dataframe *summary_1996_2025* a partir de la concatenación de los 4 dataframes que pasamos como argumentos: *total_points_by_team*, *home_goals_by_team*, *away_goals_by_team*, *total_goals_by_team*.
- Ejecuta la función y muestra los primeros valores.
- Define la función *podium(summary_1996_2025)*, que crea una gráfica de podium a partir de un dataframe donde los tres primeros valores son los equipos que componen el podium. La forma de podium se puede conseguir con un diagrama de barras, cada barra con un color, donde el primer equipo está en medio, el segundo a la izquierda a menor altura, y el tercero a la derecha a menor altura, sin etiquetas en los ejes. Sobre cada barra pondréis el nombre del equipo.
- Ejecuta la función.

Ej 7. (0,6p) Grafo

- Define la función *graf(data, selected_teams)*, donde *data* es el dataset y *selected_teams* es una lista de los 5 equipos que han conseguido mejor puntuación acumulada. Filtrarás el dataset con las filas que el equipo local y el equipo visitante estén en la lista de seleccionados. Con la librería *networkx* puedes hacer un grafo de conexiones. Por ejemplo, entre *Ath Bilbao* y *Real Madrid* hay 30+30 conexiones (Ath Bilbao ha jugado 30 veces como local y 30 veces como visitante). Puedes crear dos flechas, o bien agruparlo y poner una línea sin flecha que una Ath Bilbao y Real Madrid. Al lado de la línea tiene que haber el número de conexiones.
- Ejecuta la función y genera el grafo.

Ej 8 (2p). Proyecto Python. Organización del código. Modularidad. Capturas de pantalla

Una vez hayas conseguido la funcionalidad de los 7 primeros ejercicios, insistimos ahora en cómo tienes que organizar el código. La estructura del proyecto tiene que ser:

```
- src/
  - main.py: Fichero principal. Ejecuta los diferentes ejercicios de
  - exercises/: contiene las funciones de los diferentes ejercicios.
    - ex1.py, etc: Contiene las funciones que definimos en el ejercicio
    - ...
  - data/: el dataset
  - img/: imágenes generadas
- tests/:
  - tests_ex6.py: sólo se te pedirá test sobre el ejercicio 6
- doc/: documentación generada
- screenshots/: contiene capturas de pantalla donde se comprueba la au
- requirements.txt: librerías necesarias
- README.md: Información sobre el proyecto, cómo ejecutarlo, cómo comp
generar la documentación y ejecutar los tests.
- LICENSE: fichero de licencia
```

En el fichero *main.py* se importarán como módulos las funcionalidades de los diferentes ejercicios. La idea es que toda la funcionalidad esté en los módulos, y el fichero *main.py* sirva como punto de entrada para leer las opciones y ejecutar los módulos/ejercicios.

Se ejecutará de forma secuencial lo que se pida en los diferentes ejercicios. El fichero *main.py* admite el argumento *-h/--help* de ayuda, y el argumento *-ex* para ejecutar los ejercicios de forma incremental. Se entiende que para ejecutar el ejercicio 5 se han de ejecutar previamente los 4 primeros ejercicios. Así pues, *python main.py -ex 5* significa ejecutar los 5 primeros ejercicios. Puedes utilizar directamente la librería *sys*, o bien alguna otra posibilidad (*getopt* o *argparse*, que es la que se utiliza en la solución).

Los diferentes *prints()* que se piden estarán en el *main.py* (de manera que en los módulos/ejercicios sólo habrá la definición de las funciones). Se pide una captura de pantalla de la estructura de carpeta, donde se vea vuestro nombre, y captura de pantalla de cómo has ejecutado correctamente el proyecto.

NOTA: los diferentes módulos sólo incluirán estrictamente las librerías necesarias (el proceso de linting os informará de las librerías sobrantes).

Ej 9 (0,5p). Linter con Pylint

El código tiene que seguir la guía de estilo de Python (PEP8), exceptuando los

El código tiene que seguir la guía de estilo de Python (PEP8), exceptuando los casos en que hacerlo complique la legibilidad del código. Haz el análisis estático del código con `pylint` y limpia los errores de estilo. Se puede utilizar `.pylintrc` para adaptarlo a vuestro criterio. Importante poner en el *README* cómo se comprueba el linting. Captura de pantalla de la comprobación del linting, donde se vea vuestro nombre (no es preciso conseguir un 10, según vuestro criterio).

Ej 10 (1p). Documentación

Generad la documentación con `pydoc` (o alguna otra librería si lo consideráis más apropiada). Todas las funciones han de tener *docstrings*. La carpeta `doc/` estará en la raíz del proyecto. Importante poner en el *README* cómo se genera la documentación. Captura de pantalla de la documentación, donde se vea vuestro nombre.

Ej 11 (1p). Tests

Se pide un solo test que servirá para comprobar el funcionamiento correcto de la función `fun_total_goals` definida en el ejercicio 6. Importante poner en el *README* cómo se ejecutan los tests. Captura de pantalla de la ejecución del test en vuestra máquina, y donde se vea vuestro nombre.

Si tenéis más tiempo se puede hacer una suite de tests más completa y hacer un informe de la cobertura de vuestra suite de tests.

Ej 12 (1,5p). README.md, requirements.txt, LICENSE

Incluirás el fichero *README* (formato txt o markdown) con la siguiente información:

- vuestro nombre y título del proyecto
- estructura y organización de carpetas
- instalación del proyecto a partir de *requirements.txt*, en un entorno `venv` limpio.
- ejecución del proyecto
- comprobación del análisis estático (linting)
- generación de la documentación
- comprobación de los tests
- ficheros *requirements.txt* (incluir sólo las librerías asociadas al proyecto, no

es preciso las librerías asociadas al linting, documentación, tests) y *LICENCE* (licencia bajo la que se distribuye el código, podéis escoger la que queráis).

- Comandos para subir el proyecto a Github.

Este fichero es el primero que mirará el profesor a la hora de corregir, y espera encontrar una información clara y completa.

Tendréis que entregar un zip con el contenido del proyecto, siguiendo la estructura de carpetas que se propone.

NOTA: no incluir en el fichero de entrega las carpetas *.venv* o similares que ocupan mucho espacio. El proyecto no debería ocupar más de 5 MB (en función de las imágenes y capturas).

Rúbrica

Funcionalidad (4 p)

- Ejercicio 1 (0,4 p)
- Ejercicio 2 (0,6 p)
- Ejercicio 3 (0,6 p)
- Ejercicio 4 (0,6 p)
- Ejercicio 5 (0,6 p)
- Ejercicio 6 (0,6 p)
- Ejercicio 7 (0,6 p)

Aspectos formales del proyecto Python (6 p)

- Proyecto Python. Organización del código. Modularidad (1,5 p)
- Capturas de pantalla, carpeta img/ (0,5p)
- Linter con Pylint (1 p)
- Documentación (1 p)
- Tests (1 p)
- README.md, requirements.txt, LICENSE (1 p)